Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M2 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

Abstract*—Modern data pipelines increasingly depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-crafted declarative rules. Recent work has started to introduce machine learning (ML) anomaly detectors inside ETL workflows, but systematic, apples-to-apples comparisons between rule-based, classical ML, and deep learning approaches remain scarce—especially when evaluated within a realistic pipeline and broken down by the type of anomaly being injected. This paper presents an open-source ETL pipeline in which anomaly detectors from different families share one `fit/predict/score` interface and are scored on the same pipeline path: statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM), with deep learning detectors (Autoencoder, LSTM-Autoencoder) planned behind the same interface. I evaluate the detectors, over five seeds on a stratified held-out split, on four datasets (and, for generality, on 100 labelled problems and 42 real tables with injected anomalies): Kaggle Credit Card Fraud, a rare-anomaly Cardio dataset, a common-class Water Quality dataset, and a controlled synthetic dataset generated with Faker in which six distinct anomaly types are injected at controlled rates. Beyond aggregate precision, recall, F1, ROC-AUC and PR-AUC, I report a per-anomaly-type recall and precision, precision-recall curves, and a sweep of the injection severity. The aggregate metrics make the families look interchangeable; the per-type view shows each has a narrow, largely non-overlapping competence, so broad coverage needs several families rather than the single best one. My goal is a reproducible, pipeline-integrated benchmark that answers the practical question: *which family of detectors should I use, for which kind of data quality problem, at what operational cost?

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications. Organizations increasingly manage data as an asset that must be governed [1], and the value delivered downstream is bounded by the quality of the data that reaches the destination. This is why data quality has long been a first-class concern for data engineers, and why automated, continuous quality control is a capability rather than an option.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [2]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [3]–[5]. The existing literature, however, suffers from three gaps.

First, the pipelines I reviewed are mostly presented as architectures. Gopa [3] reports aggregate gains taken from other implementations and releases no code, and the abstract of Marupaka and Rangineni [4] announces accuracy gains without naming a dataset or a baseline. When an experiment is reported, it exercises a single algorithm on a single dataset: Sokkula and Vuppala [5] run Isolation Forest on Credit Card Fraud after oversampling the training data with SMOTE, and report an ROC-AUC of 0.51 with a fraud recall of 0.02. A near-random score on one dataset offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of a wide range of detectors—but these benchmarks evaluate algorithms on their own, outside of an ETL context. ADBench reports training and inference times for each algorithm, but it has no declarative rule-based baseline of the kind that dominates production systems, and it does not measure what a detector costs inside a running pipeline (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors

excel at different kinds of anomalies [8]. ADBench does look at anomaly types, but as geometric types generated on synthetic data (local, global, dependency and clustered anomalies), not as the data quality faults an ETL pipeline meets. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Contributions.: This paper addresses these three gaps. I present:

1) **A reproducible ETL pipeline** in which rule-based and classical ML anomaly detectors share one interface and are scored on the same extract–transform–detect path, released as open-source code, with deep-learning detectors planned behind the same interface.
2) **A head-to-head benchmark** of statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM) on four datasets, with a stratified held-out split, five seeds, and the ML operating point read from the data, reporting precision, recall, F1, ROC-AUC, PR-AUC, precision-recall curves, and detection-stage latency (fit, predict and score; extraction, transformation and loading are not timed).
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at a tunable severity, I produce a method-by-type recall and precision matrix and test its sensitivity under a severity sweep. The families turn out to have narrow, largely non-overlapping competence that the aggregate metrics hide.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes my pipeline architecture, the detectors I compare, and the datasets and metrics I use. Section IV reports experimental results. Section V discusses when rules, classical ML, and deep learning are each the right tool, and highlights practical trade-offs in latency and operational cost. Section VI concludes and outlines future work.

II. Related Work

To be written. Planned subsections:

- Data quality in ETL pipelines (Khatri & Brown [1]; Data Quality Survey [2]; pipeline tools survey [9])
- Rule-based and declarative approaches (Great Expectations, Deequ, Soda [2])
- Classical ML for anomaly detection (Isolation Forest, LOF, One-Class SVM; Goldstein & Uchida [8]; PyOD [7])
- Deep learning (Autoencoder, LSTM-Autoencoder)
- Existing benchmarks: ADBench [6], which covers 30 algorithms, 57 datasets, geometric anomaly types and standalone run times, and what it leaves out (rule-based baselines, ETL integration, data quality fault categories)
- ML in ETL pipelines: Gopa [3], Marupaka & Rangineni [4], Sokkula & Vuppala [5] as illustrative limitations

III. Methodology

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and deep models can later be dropped in without changing the harness.

B. Rule-based detectors

This stage compares three rule-based baselines. All three define a band of "normal" values per column and flag a row when it leaves the band on at least one column; they differ in how the band is estimated and in how robust that estimate is to the very outliers being sought.

Z-score standardizes each numeric column on the fitted data and flags a row when any column exceeds $|z| > 3$. The threshold follows the empirical rule: under an approximately normal distribution about 99.7% of values lie within 3σ, so beyond it is rare by construction. Its weakness is that the mean and standard deviation are themselves inflated by extreme values, so a large outlier can mask others.

IQR learns Tukey's fences $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ per column, where the interquartile range spans the central 50% of the data, and flags a row falling outside any fence. Quartiles are robust: unlike the mean and standard deviation they barely move when extreme values are added, so the fences are not distorted by the outliers they are meant to catch. The 1.5 multiplier corresponds to about 2.7σ under normality, isolating a comparable tail.

Great Expectations encodes a declarative expectation suite [2] and makes no distributional assumption: a not-null expectation on every column and a range expectation per numeric column, with bounds set to the 0.1th and 99.9th percentiles of the fitted data. Using inner percentiles rather than the observed min/max is what turns the range check into a usable rule instead of a no-op on the fitting data. A row is flagged when it violates at least one expectation; its score is the fraction of the suite it violates.

The continuous scores of the statistical detectors are the worst standardized deviation (Z-score) and the largest fence exceedance in IQR units (IQR). The axis of difference between the three is thus robustness and the assumption made about the shape of the data, which is precisely what the learned models of the next stage are meant to handle better.

C. Classical machine-learning detectors

The second family replaces the per-column band with a learned model of the joint feature space, so that a row can be flagged for an implausible *combination* of otherwise-normal values. Three unsupervised detectors are taken from PyOD [7] rather than reimplemented: *Isolation Forest*, which scores a row by how few random axis-aligned splits are needed to isolate it; *Local Outlier Factor* (LOF), which compares a point's local density to that of its neighbours; and a *One-Class SVM*, which fits a boundary enclosing the bulk of the training data. All three receive the same preparation: non-numeric columns are dropped, missing cells are median-imputed, and every feature is standardized, since LOF and the One-Class SVM are distance-based. The PyOD `decision_function` provides the continuous score; the boolean flag uses a *contamination* equal to the anomaly prevalence of the training split (clamped to PyOD's valid range), so each model marks about as many rows as the data actually contains rather than a fixed share. Contamination sets only the flag threshold and leaves ROC-AUC and PR-AUC unaffected.

D. Synthetic per-anomaly-type dataset

The real datasets carry a single binary label that mixes every kind of defect together, so they cannot say *which* problem a detector is good at. I therefore generate a controlled dataset of $12{,}360$ customer records with Faker and NumPy, in which income is correlated with balance and transaction size and account age is spread over three years. Into this clean population I inject six anomaly types, each at a rate of 3% and each row tagged with its category: *missing values* (one or two cells nulled), *exact duplicates* (a clean row copied), *univariate outliers* (one numeric field pushed far out of range), *multivariate outliers* (every field individually in range but the combination implausible, e.g. an 18-year-old with a top-percentile balance and income), *temporal drift* (recent-signup rows whose income and balance distribution has shifted upward by half), and *invalid formats* (malformed e-mail, out-of-domain country code, impossible age). Because the category of every injected row is known, I report, per method and per type, both the detection rate (recall) and the precision on a stream made of clean rows plus that one type. The injection magnitudes are exposed as a single *severity* multiplier so the choice of magnitude can be swept rather than fixed (Section IV-H). I also add a time-aware detector, *drift*, that learns a per-feature baseline from the oldest rows and flags recent rows that both sit in a window whose mean has moved and lean in the direction of that move; it is run on the synthetic dataset only, as the other detectors ignore the row order.

E. Datasets

I evaluate on four datasets. *Credit Card Fraud* [10] (Kaggle; $283{,}726$ transactions after de-duplication, 30 numeric features, 0.17% labelled fraud) is the standard strongly imbalanced benchmark, but its positive class mixes every kind of defect. The *synthetic* dataset above ($12{,}360$ rows, 17.5% anomalous) carries the per-type labels. *Cardio* (a cardiotocography anomaly dataset as packaged for outlier-detection benchmarking; $1{,}831$ rows, 21 standardized features, 9.6% anomalies) is a *genuinely imbalanced* real dataset [6], [11]: the anomalies are rare and distributionally distinct, the regime the unsupervised detectors are built for. *Water Quality* [12] ($3{,}276$ samples, 9 numeric features) is the opposite case: the target class ("not potable") is common (61%) rather than rare, and the data has genuine missing values, so it probes the detectors outside their regime.

F. Evaluation protocol

Each dataset is split $70/15/15$ (train/validation/test), stratified on the label. For the synthetic dataset, rows are grouped by customer identity before stratification so an original and its exact copy cannot cross partitions. Detectors are fitted on the train split and every reported number is computed on the held-out test split; the validation split is reserved for later tuning. The ML contamination is read from the train split's prevalence. This is a label-informed operating-point choice, although estimator fitting itself does not use the labels. The real-data benchmark also permits a contamination value specified before evaluation, independent of every split's labels. This label-free preset forms a distinct comparison protocol; the numerical results in this manuscript retain the train-prevalence baseline. In the sensitivity sweep, contamination follows a fixed grid and no value is selected from the test metrics. Threshold or hyperparameter selection for a later confirmatory experiment would use training or validation data only. The whole procedure is repeated over five random seeds and I report the mean and standard deviation. This spread measures the sensitivity to the split and to model randomness on a fixed dataset; it is not a confidence interval for performance on new data. LOF and the One-Class SVM are run on Water Quality, Cardio and the synthetic dataset but not on Credit Card Fraud, where LOF's superlinear scoring and the One-Class SVM's quadratic training were expected to be too costly at 284k rows; this was not measured.

Because the detectors emit both a threshold decision and a graded score, I report threshold metrics (precision, recall, F_1) alongside ROC-AUC and PR-AUC (computed as average precision, not trapezoidal area). On imbalanced data PR-AUC is the more informative metric: the false-positive rate underlying ROC-AUC is diluted by the large true-negative pool and stays optimistic, whereas precision reacts directly to false alarms. I keep both because PR-AUC's random baseline equals the prevalence and is not comparable across datasets, while ROC-AUC's is always 0.5 and is. I also report the fraction of rows flagged and the in-pipeline latency. Because a single flag threshold hides the rest of the trade-off, I additionally draw the full precision-recall curve of every detector (Fig. 2) and sweep Isolation Forest's contamination across each dataset (Table V). The deep-learning detectors (Autoencoder, LSTM-Autoencoder) are the object of the next stage.

G. Verification and reproducibility

The experimental code is checked at three levels. A routine test suite exercises numerical contracts on small, generated tables and measures line and branch coverage across the pipeline and experiment scripts. Its regression cases include partition disjointness by customer identity, training-only preprocessing, the exclusion of labels from detector inputs, score direction and index alignment, undefined binary metrics, and the construction of each synthetic anomaly type. Separate command-line tests run the four figure and table generators on temporary data. They check the output schema without comparing images pixel by pixel or imposing a fixed wall-clock duration.

A second, manually invoked level asks whether these checks can fail. Mutation testing generates small source-level changes (a reversed comparison, an altered constant, a modified message) and counts how many make the routine suite fail. I apply it to the modules that compute a result and to the experiment scripts. In addition, twenty-three deliberate scientific defects are applied one at a time to a temporary copy of the code, and a named test must fail on each: a scaler refitted on the rows being scored, the label passed to the detector as a feature, a contamination threshold read from the test labels, a training row copied into the test partition, an inverted score orientation, ranking metrics computed from the binary flags, a seed aggregate that keeps only the first seed, a detector missing from the results table, and a few others.

A third level uses the locally available full datasets. A separate command compares, for each of the three research CSV files, the file checksum, the raw and post-deduplication row counts, the number of anomalies and the positive-class value with an archived manifest. An absent file is reported as not run, and the command as a whole passes only if all three files pass. The scripts record the dataset fingerprint, software versions, random seeds, split policy and output names alongside their results. For each real-data seed, an archive stores the positions of the train, validation and test rows after deduplication; a capped fit also records the selected training positions. Archived tables and figures are checked against their manifest, and the six numerical tables in this manuscript are compared with the archived CSV values at the precision shown. This establishes traceability between the reported values and the recorded experiment; it does not establish that another machine will reproduce the same timings.

A fourth part turns these checks into a release verdict. Each profile (routine suite, mutation campaign, fault injection, real-data check, coverage of the checking scripts themselves, static typing, and a controlled reproduction) writes a small evidence record tied to the source revision, to a clean working tree, to a fingerprint of the code that computes results (the pipeline and the experiment scripts, not the checking scripts) and to the software versions. A release script recomputes every status from the raw results instead of reading it from the record, requires that all profiles describe the same revision and compatible environments, and returns *certified*, *incomplete* or *rejected*; figures quoted in this manuscript are accepted only if the run they come from is not older than a change to the code. The real-data check can also run on a hosted Linux machine: it downloads the three public files, compares each SHA-256 value with the archived manifest before using it, and refuses any other file. The controlled reproduction replays the cardiotocography benchmark against the archive, twice in the same environment and once in an environment rebuilt from a recorded, hash-pinned lock; on another operating system the metrics agree statistically (largest difference 6.5×10^{-5}, identical partitions) rather than bit for bit. Result files are written to a temporary name and renamed only when complete, so an interrupted run cannot leave a truncated table or figure. The verdict states where the evidence was produced and never that it was independently attested: the author wrote the code and launched every run.

I treat a failed or missing check as a limitation of the evidence, not as a successful experiment. Complete line and branch coverage and a mutation score of at least 95% over the pipeline and the scripts are the targets, and the measured values, with what they leave out, are reported with the results below. The chronological split and the window-based drift monitor have a separate protocol: their alert and delay measures are not pooled with the row-level metrics from the stratified benchmark.

IV. Experiments and Results

Benchmark tables report means over five seeds on the held-out test split (Section III-F). Seed variability is reported in the archived result tables; timing also depends on the execution environment.

A. Real fraud data

Table I reports the detectors on Credit Card Fraud. The statistical rules recover most frauds (recall 0.87–0.97) at a precision below 0.04: a fixed per-column rule cannot separate a 0.17% positive class from the tails of legitimate traffic. Great Expectations is the most selective of the rules (3% of rows) and leads PR-AUC at 0.182 ± 0.046. Isolation Forest behaves differently now that its contamination matches the prevalence: it flags only 0.17% of rows and reaches a precision of 0.20, the first usable operating point in the table, but its recall drops to 0.21 and both are seed-sensitive ($F_1 = 0.20 \pm 0.07$). It keeps the best ROC-AUC (0.934). Great Expectations took $1.3\,\mathrm{s}$ on this run, against 0.03–$0.12\,\mathrm{s}$ for the statistical rules; these times vary between runs. LOF and the One-Class SVM were not run at 284k rows, where their cost was expected to be prohibitive; I did not measure it.

B. Genuinely imbalanced real data: Cardio

Table II runs all six detectors on Cardio, where the anomalies are both rare (9.6%) and distributionally distinct. Here the unsupervised detectors work: Isolation Forest reaches $F_1 = 0.62$, PR-AUC 0.70 and ROC-AUC 0.94, and the One-Class SVM is close behind. Z-score also does well on this dataset ($F_1 = 0.61$) because the 21 features are already standardized

TABLE I
DETECTORS ON CREDIT CARD FRAUD, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. HIGHER IS BETTER EXCEPT FLAGGED ROWS AND TIME.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.870	0.021	0.926	0.078	13.4	**0.04**
IQR	0.003	**0.966**	0.007	0.911	0.018	48.6	0.16
Great Expectations	0.037	0.676	0.070	0.830	**0.182**	3.1	1.91
Isolation Forest	**0.202**	0.206	**0.203**	**0.934**	0.120	**0.2**	2.42

TABLE II
ALL DETECTORS ON CARDIO, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. 1,831 RAW ROWS (1,822 AFTER DEDUPLICATION), 9.6% ANOMALIES.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.49	**0.81**	0.61	0.93	0.61	16.6
IQR	0.21	0.90	0.33	0.78	0.30	43.4
Great Expectations	0.50	0.21	0.29	0.59	0.19	4.6
Isolation Forest	**0.64**	0.61	**0.62**	**0.94**	**0.70**	9.9
LOF	0.25	0.27	0.26	0.62	0.24	10.7
One-Class SVM	0.56	0.61	0.58	0.94	0.63	10.7

and the anomalies stand out on several of them at once. This is the counterpoint to Credit Card Fraud and Water Quality: the value of a learned detector shows when the anomalies are genuinely rare outliers, not when the positive class is a 0.17% mixed-defect label or a 61% majority. Isolation Forest and the One-Class SVM carry a wide seed spread on this small dataset (F_1 standard deviations of 0.14 and 0.17).

C. Synthetic dataset: aggregate metrics

Table III reports the six core detectors on the synthetic dataset. Aggregate metrics still compress the families together: F_1 ranges from 0.33 to 0.50 and PR-AUC from 0.33 to 0.54. Isolation Forest has the best F_1 (0.50) and Z-score keeps a perfect precision by firing only on the most extreme rows. The One-Class SVM approaches Isolation Forest on quality while taking longer in this run. Read alone, these numbers suggest the detector choice barely matters; the per-type breakdown shows the opposite.

D. Synthetic dataset: by anomaly type

Fig. 1 gives the mean recall of each method on each injected type. The structure the aggregate table hides is now clear: every method has a narrow competence.

- *Univariate outliers* are caught by every distributional method (recall 0.70–1.00) and only by those. Z-score's precision on them is 1.00: it never fires on a clean row.
- *Missing values* are caught only by the Great Expectations not-null expectations (recall 1.00, precision 0.94), because the statistical rules propagate `NaN` to a non-flag and the learned models see imputed values.
- *Exact duplicates* are invisible to every detector. The deduplication rule of the transform stage is the only thing that catches them, at precision 0.50 because it also flags each row's clean original. In this grouped split the rule is evaluated within the held-out partition; it is a stream-level reference, not a model.
- *Multivariate outliers* separate the families: Isolation Forest (0.89) and the One-Class SVM (0.90) are the only methods with real recall, since they are the only ones that model the joint distribution. Their precision on this type is only about 0.22: they cast a wide net that also catches clean rows.
- *Invalid formats* are caught mainly through the impossible numeric value they carry (IQR recall 1.00, precision 0.37; Z-score precision 1.00); the malformed string fields are missed by all.
- *Temporal drift* is the weakest column for every method (best recall 0.50, LOF). The dedicated *drift* detector scores 0 here too: at the reference magnitude the shifted values stay inside the global range, and once diluted by the many clean recent rows the window mean has moved by less than a tenth of a standard deviation on average (0.04 to 0.11 over the five test splits). The present row-level drift detector remains at zero even in the severity sweep; the windowed-monitoring extension is separate.

The practical reading is a short decision list: null-rate problems to declarative rules, single-column outliers to any statistical rule, implausible combinations to a tree- or boundary-based model, and duplicates and drift to dedicated stages the detectors do not replace.

TABLE III
ALL DETECTORS ON THE SYNTHETIC DATASET, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY.

Detector	Prec.	Rec.	F_1	ROC	PR	Time (s)
Z-score	**1.000**	0.196	0.328	**0.742**	**0.544**	**0.004**
IQR	0.584	0.396	0.472	0.675	0.452	0.007
Great Expectations	0.951	0.202	0.333	0.600	0.333	0.46
Isolation Forest	0.495	**0.499**	**0.497**	0.735	0.485	0.34
LOF	0.388	0.420	0.404	0.659	0.437	0.22
One-Class SVM	0.478	0.485	0.481	0.722	0.520	4.20

E. Third real dataset: Water Quality

Table IV runs all six detectors on Water Quality, where "not potable" is the target and covers 61% of the rows. Every detector's ROC-AUC sits between 0.45 and 0.51 and every PR-AUC lies within 0.04 of the 0.61 base rate: the unsupervised detectors are essentially at chance. This is the expected outcome and a useful negative result. When the target class is common and not distributionally distinct from the rest, "anomaly" detection has nothing to key on. The dataset still exercises the pipeline on real missing data: Great Expectations flags 40% of rows on its not-null expectations, but the nulls here are uncorrelated with potability, so that recall buys no precision.

F. Precision-recall curves and the operating point

Fig. 2 draws the full precision-recall curve of every detector on each dataset's test split. The curves make the aggregate

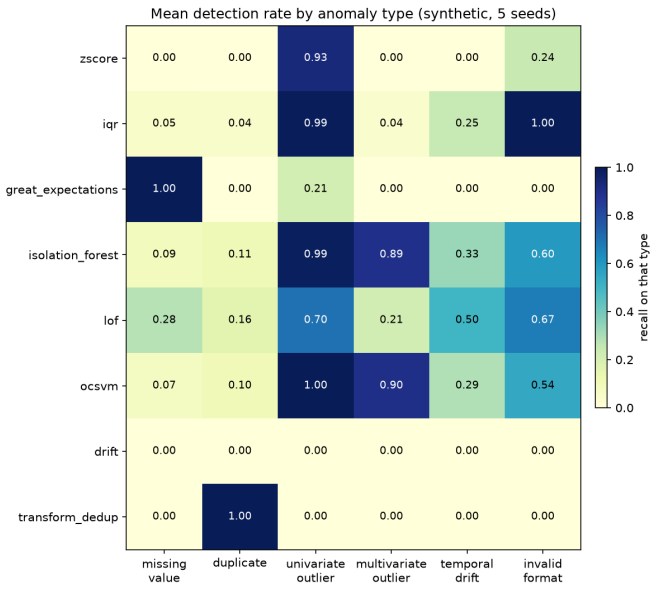


Fig. 1. Mean detection rate (recall) over five seeds of each method on each injected anomaly type, synthetic test split. Darker is higher. `transform_dedup` is the deduplication rule of the transform stage, shown for reference.

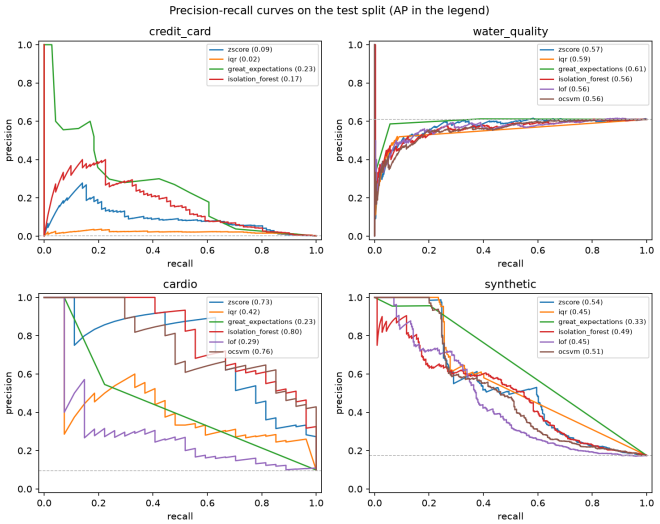


Fig. 2. Precision-recall curves on the test split, one panel per dataset, average precision in the legend. The dashed line is the base rate.

TABLE IV
ALL DETECTORS ON WATER QUALITY, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TARGET: "NOT POTABLE", BASE RATE 0.61.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.41	0.02	0.05	0.46	0.58	3.5
IQR	0.57	0.09	0.16	0.49	0.60	10.0
Great Expectations	0.62	0.41	0.49	0.51	0.61	40.1
Isolation Forest	0.57	0.46	0.51	0.45	0.57	49.7
LOF	0.58	0.46	0.52	0.46	0.58	48.7
One-Class SVM	0.57	0.45	0.50	0.46	0.57	48.4

TABLE V
ISOLATION FOREST UNDER A CONTAMINATION SWEEP, TEST SPLIT, ONE SEED. "PREV." IS THE TRAINING-SPLIT PREVALENCE, THE VALUE THE BENCHMARKS USE.

Dataset	Contam.	Prec.	Rec.	Flag.%
Credit Card Fraud	0.001	0.37	0.23	0.10
Credit Card Fraud	0.10	0.01	0.90	10.2
Credit Card Fraud	prev. 0.002	0.28	0.32	0.20
Cardio	0.01	1.00	0.33	3.3
Cardio	0.20	0.46	0.93	19.7
Cardio	prev. 0.096	0.72	0.67	9.1
Water Quality	prev. 0.5	0.58	0.49	51.6
Synthetic	prev. 0.175	0.53	0.51	16.8

tables concrete: on Cardio the Isolation Forest and One-Class SVM curves stay high (AP 0.80 and 0.76); on Water Quality every curve hugs the 0.61 base-rate line; on Credit Card Fraud Great Expectations and Isolation Forest dominate but neither clears a precision of 0.4 except at negligible recall. Table V makes the point numerically for Isolation Forest: as the contamination moves from 0.001 to 0.20, its Credit Card Fraud precision falls from 0.37 to 0.008 while recall rises from 0.23 to 0.92; on Cardio the same sweep trades a precision of 1.00 at recall 0.04 for a recall of 0.93 at precision 0.46. The estimator is fixed; only the threshold moves. Reading the prevalence off the training split lands near the knee of the curve on the datasets where a knee exists.

G. Fixed-contamination sensitivity analysis

The train-prevalence operating point uses labels from the training partition. To measure its influence, I reran the three real datasets with the same five seeds and partitions, replacing that value by a constant contamination of 0.02 for the ML detectors. Table VI shows Isolation Forest, the only learned detector run on all three datasets. The ranking score and its average precision are unchanged; only the binary decision threshold moves. On Credit Card Fraud, the fixed setting raises recall from 0.206 to 0.682 but lowers precision from 0.202 to 0.055. On Cardio, it raises precision from 0.639 to 0.983 but lowers recall from 0.615 to 0.267. The Water Quality target is a majority class, so a 2% outlier budget leaves almost all positive rows unflagged. The fixed value was chosen before these comparator runs but after the baseline results were available. These numbers are therefore a post-hoc sensitivity analysis, not a new confirmatory estimate on an untouched test set.

H. Detection rate versus injection severity

The synthetic magnitudes are a design choice, so I sweep them. A single *severity* multiplier scales the univariate factor, the multivariate percentile extremity and the temporal-drift multiplier (1.0 is the dataset used above). Fig. 3 plots per-type recall against severity. Univariate outliers are already saturated at severity 0.5. Multivariate outliers show a threshold: Isolation Forest and the One-Class SVM climb to recall 1.0, while IQR jumps from 0.04 to 1.0 once the injected percentile crosses

TABLE VI
ISOLATION FOREST WITH TRAIN-PREVALENCE AND FIXED 0.02 CONTAMINATION, SAME FIVE REAL-DATA SPLITS. ENTRIES ARE MEANS; "FLAG" IS THE FRACTION OF TEST ROWS PREDICTED ANOMALOUS.

Dataset	F_1 train	F_1 fixed	Flag train%	Flag fixed%
Credit Card Fraud	0.203	0.102	0.168	2.063
Cardio	0.618	0.406	9.854	2.701
Water Quality	0.512	0.023	49.675	1.789

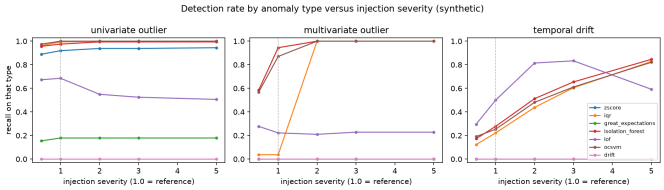


Fig. 3. Per-type recall against injection severity (1.0 is the reference synthetic dataset), mean over three seeds. The dashed line marks severity 1.0.

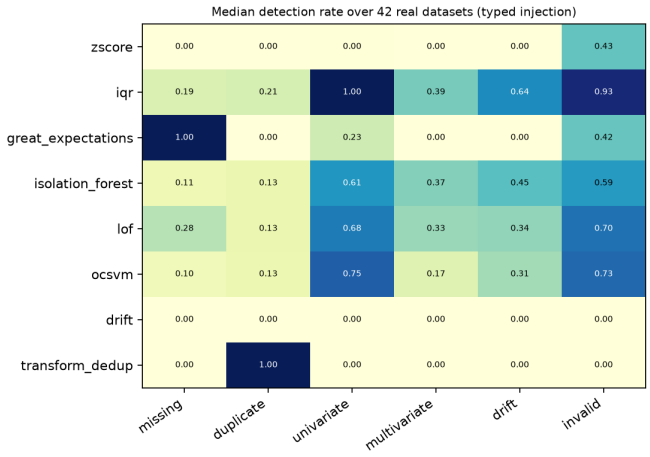


Fig. 4. Median detection rate over 42 real datasets with typed injection. Compare with Figure 1.

its fence. Temporal drift becomes easier for IQR, Isolation Forest and One-Class SVM (roughly 0.2–0.3 recall at severity 1.0 and 0.6–0.7 at severity 3), while LOF peaks earlier and then falls. The row-level drift detector does not respond in this sweep. The ranking therefore changes with severity, limiting claims based on one injection magnitude.

I. Typed injection on real datasets

The synthetic matrix of Section IV-D comes from a single generator. To see whether its pattern depends on that generator, the six anomaly types were injected into 42 real tables from the UCI repository (open licence, pinned by checksum; `verification/dataset_pool.md`). For each table only the continuous columns are kept, and for labelled sets only the normal class, so that the original anomalies never enter the evaluation. The injection follows the synthetic one (three percent of rows per type, same split, seeds and detector settings), with numeric versions of the types: a value several standard deviations away, a pair of correlated columns pushed to opposite tails, a shift of three columns for the most recent rows, a sign flip, a sentinel or a thousandfold scale error. The protocol and five hypotheses were written before any run (`verification/typed_injection.md`).

Two rows of the synthetic pattern hold on every table: Great Expectations finds all missing values and the deduplication rule all duplicates. The rest does not carry over. Three of the five hypotheses are not supported (Great Expectations and deduplication aside, the specialisation, the multivariate gap and the difficulty of drift). On multivariate outliers the median detection rate is 0.37 for Isolation Forest and 0.17 for the one-class SVM, against 0.89 and 0.90 on the synthetic data. The interquartile fences reach the best detection rate on several types, but they also flag a median of 22% of the clean rows (up to 99% on some wide tables), so the detection rate alone favours them. The z-score detects almost nothing on univariate outliers (median 0.00 against 0.93 synthetically). A check made after seeing this, and therefore exploratory, shows why: the invalid rows of the training split inflate the standard deviation, and without them the z-score detects every univariate outlier on all 42 tables. The result says more about non-robust statistics on a contaminated training set than about the z-score itself.

The two hypotheses on stability hold. The mean rank correlation between the method-by-type matrices of two tables is 0.83, and it is lowest for drift (0.70) and multivariate outliers (0.74), though only slightly below the structural types (0.75 to 0.82). Part of this stability is mechanical, since many cells are close to 0 or 1. The detection rate of Isolation Forest on univariate outliers falls with the number of continuous columns (rank correlation -0.96 over the 42 tables). The anomalies are injected, not natural, at one severity, and the deep models are not part of this matrix.

J. Labelled problems of the dataset pool

The three real datasets of this section are few. The pool of 152 open tables (`verification/dataset_pool.md`) holds 100 problems whose anomalies are already labelled: 36 from ADBench and 64 from the UCI repository, where the rarest class (at least 20 rows) is the anomaly and the other classes are normal; a class above 10% of the rows is resampled to 10%. I ran the six detectors of the main benchmark on all of them with the protocol of Section III-F, written down before the run (`verification/labelled_pool.md`): one normalisation for every table, a stratified 70/15/15 split, five seeds, contamination equal to the training prevalence and default settings. LOF is skipped above 50,000 training rows and One-Class SVM above 20,000, so they cover 92 and 84 problems; the skips are listed in the archive. All 100 runs completed.

No detector is clearly ahead. One-Class SVM has the highest median PR-AUC (0.240) and Isolation Forest the best median rank, but their intervals overlap with each other and with the Z-score, so the hypothesis that no method is universal is supported. The best method differs from one problem to the

TABLE VII

MEDIAN OVER THE LABELLED PROBLEMS OF THE SEED-AVERAGED TEST METRICS, WITH A 95% BOOTSTRAP INTERVAL OVER THE PROBLEMS, AND THE MEDIAN RANK OF THE METHOD ON PR-AUC (1 IS BEST).

Detector	Problems	PR-AUC	ROC-AUC	Rank
Z-score	100	0.203 [0.164, 0.279]	0.626	3
IQR	100	0.144 [0.117, 0.191]	0.556	4
Great Expectations	100	0.115 [0.105, 0.123]	0.508	5
Isolation Forest	100	0.215 [0.158, 0.293]	0.677	2
LOF	92	0.165 [0.153, 0.237]	0.588	3
One-Class SVM	84	0.240 [0.177, 0.307]	0.670	3

next: Isolation Forest is first on 26 problems, LOF on 24, the IQR rule on 16, the Z-score on 14, One-Class SVM on 13 and Great Expectations on 7. Learning does help on median: Isolation Forest is above the Z-score on 67 problems and above the IQR rule on 62, but the gain is small (about 0.01 to 0.04 of PR-AUC). The median ROC-AUC stays between 0.51 and 0.68, and on 30 problems no detector reaches 0.6. These anomalies are mostly not outlying in the sense these detectors assume, which is expected for labels taken from a rare class rather than from a data-quality defect. Great Expectations, built for declared constraints, is close to chance here.

Two of the five hypotheses are not supported. The advantage of Isolation Forest over the Z-score was expected to vanish when anomalies are more than 20% of the rows; it is smaller there (+0.013 of median PR-AUC on 12 problems) but still positive, as it is below 5% (+0.037, 19 problems) and in between (+0.009, 69 problems). The IQR rule was expected to flag more than three times the anomaly prevalence; its median is 2.3 times, the Z-score's 1.7, and the learned detectors flag about the prevalence by construction. The order of the methods is the same for ADBench and for UCI (rank correlation 0.94), although the level is much lower on UCI (best median PR-AUC 0.19 against 0.40). Nineteen sources appear in both lists with different derivations, so the problems are not independent, and 10% resampling fixes the median prevalence at 0.10. The ADBench half is closer to the usual benchmark and the UCI half is a harder, rule-derived labelling; I did not tune any detector, so the numbers describe default settings.

K. Deep models in the per-type matrix

The matrix of Figure 1 has no learned reconstruction model. I added two rows on the same synthetic data and the same identity-grouped splits and seeds, with the protocol and four hypotheses written beforehand (`verification/deep_typed.md`): the sklearn autoencoder of the registry, and the LSTM autoencoder of Section IV-M on windows of the ten preceding customers ordered by signup date. Unlike the Credit Card pilot, both models were trained until their validation loss stopped improving (patience of 5, at most 300 epochs), and all ten trainings stopped before the cap (60 to 178 epochs). The other rows were recomputed by the same code and equal the published matrix.

The deep rows do not change the picture. The autoencoder detects multivariate outliers (0.83 ± 0.10) about as well as

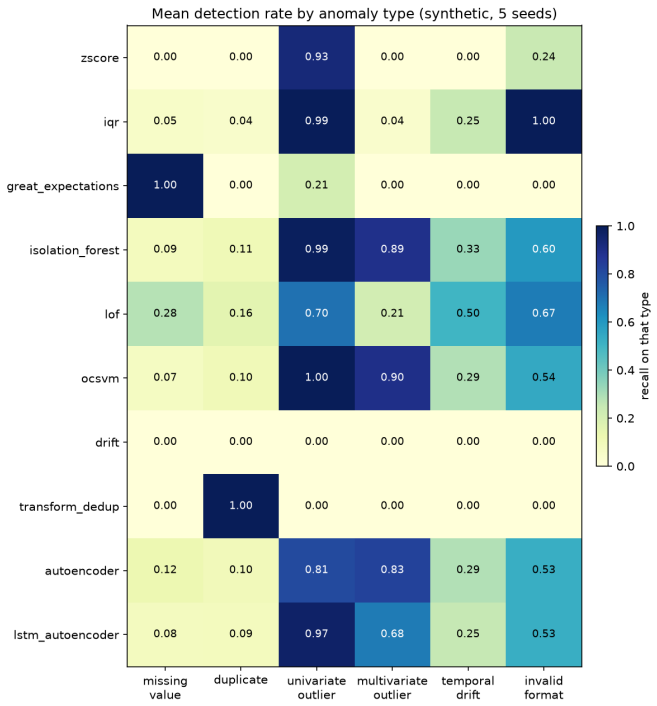


Fig. 5. Mean detection rate by anomaly type with the two deep rows added (synthetic, five seeds). Compare with Figure 1.

Isolation Forest (0.89) and One-Class SVM (0.90), as expected for a model that reconstructs correlations, and the LSTM is a little lower (0.68 ± 0.12). Both are close to the learned detectors on the structural types, and on missing values and duplicates they are no better than the best row-level detector. The hypothesis I expected to matter, that the sequence model would catch temporal drift better than every row-level detector, is not supported: its recall is 0.25 ± 0.07 against 0.50 for LOF, 0.33 for Isolation Forest and 0.29 for the autoencoder. Three of the four hypotheses hold (structural types, convergence, multivariate outliers). The synthetic customers are independent rows, so the signup order carries a sequence only for the drifted rows, which are the most recent ones; a window of ten rows that are mostly clean and have shifted values in only some of the columns gives the LSTM little to learn from. This says that this construction of drift is not a sequence problem, not that sequence models cannot detect drift. The false-positive rate on clean rows is about 12% for both models. With five seeds the standard deviations are indicative and none of these comparisons is a test.

L. Chronological pilot on Credit Card

The results above use a random stratified split. Of the three real datasets, only Credit Card has a time variable (seconds since the first transaction, about two days); Cardio and Water Quality have none, so their row order says nothing about time. I therefore ran one small pilot on Credit Card, with the protocol written down before any score was computed on held-

TABLE VIII
AVERAGE PRECISION ON THE TEST SPLIT, STRATIFIED CONTROL VERSUS CHRONOLOGICAL SPLIT (95% BLOCK-BOOTSTRAP INTERVAL), AND THE SHARE OF FUTURE ROWS FLAGGED AT THE TRAINING-PREVALENCE THRESHOLD.

Detector	PR, stratified	PR, chronological	ROC, chron.	Flag %
Z-score	0.078 ± 0.015	0.048 [0.020, 0.087]	0.935	3.8
IQR	0.018 ± 0.003	0.013 [0.007, 0.023]	0.917	47.1
Great Expectations	0.183 ± 0.047	0.045 [0.013, 0.088]	0.758	3.5
Isolation Forest	0.110 ± 0.039	0.035 [0.016, 0.066]	0.936	0.08

out data (`verification/temporal_pilot.md`). It is a separate experiment and does not replace the benchmark.

The rows are the same cleaned rows, and the detectors are the same four with default settings, but the clock variable is removed from the features. The chronological split uses the usual $70/15/15$ ratios by time: the detectors are fitted on the first 198,609 rows (366 frauds) and scored on the last 42,557 rows (52 frauds, 0.12%). The control uses the stratified split of the main benchmark with the same rows and features, over five seeds. Intervals for the chronological arm come from a block bootstrap over 30-minute blocks of the test period.

Average precision is lower under the chronological split for all four detectors, but the interval excludes the stratified mean only for Great Expectations and Isolation Forest; for the Z-score and the IQR rule the difference is within the uncertainty of 52 test frauds. This exclusion criterion was fixed in advance as an exploratory check. It is not a confidence interval or a test for the difference between the two protocols: the bootstrap reflects only the chronological arm, the stratified mean is itself uncertain (five seeds), and the share of frauds changes between partitions (0.18% in training, 0.12% in the test period), so part of the gap may come from prevalence alone, which this pilot does not separate from the effect of time. The best detector also changes: Great Expectations leads the stratified control, while the Z-score has the highest chronological point estimate, with overlapping intervals. Isolation Forest keeps a high ROC-AUC (0.936) but flags only 0.08% of the future rows and none of the 52 frauds: the threshold set from training scores does not carry over to the later period. This is a single two-day dataset with few positives, so it indicates a direction and an order of magnitude, not a general rule.

To check the window monitor with a known change, a mean shift of δ training standard deviations was added to five features from the middle of the test period. The alert threshold was fixed from the validation period before the test (the largest validation window score, 0.91). Shifts of 1 and 2 standard deviations were detected in the second window after the change (573 seconds), and a shift of 0.5 was not detected; no window before the change raised an alert (22 windows). Credit Card has no labelled change in distribution, so I make no claim about its natural drift.

M. LSTM autoencoder pilot

A last pilot asks whether the preceding rows of the transaction stream carry a fraud signal that a row-level autoencoder cannot see. Credit Card has no cardholder identifier, so the window is the global stream of the ten previous rows, not a customer's history. The protocol was fixed beforehand (`verification/lstm_pilot.md`). The comparison uses the chronological partitions of Section IV-L, the same 50,000 training rows and a CPU budget of at most 20 epochs for both models; the architecture (window of 10 rows, 32 units) was not tuned, and the validation rows served only for early stopping on an unlabelled reconstruction loss.

As a check before reading the real data, a synthetic stream whose anomalies break the continuity of an autoregressive signal was scored by both models: the LSTM reaches an average precision of 0.875 against 0.006 for the row-level autoencoder. On Credit Card, over five initialisation seeds, the LSTM reaches an average precision of 0.037 ± 0.006 on the 52 test frauds against 0.024 ± 0.010 for the row-level autoencoder (ROC-AUC 0.926 against 0.841). The paired block-bootstrap interval of the difference excludes zero for the main seed ($+0.021$, $[+0.005, +0.044]$) but not for the other four, where the difference is positive for three seeds and negative for one. This criterion was set in advance as an exploratory check, not a formal test. The LSTM is comparable to Isolation Forest (0.035) and Great Expectations (0.045) on the same rows, not clearly better. None of the five trainings converged within the budget: the validation loss was still decreasing at the 20th epoch. As for the other detectors, the threshold taken from training scores flags almost none of the frauds in the later period (recall at most 0.02). The result is therefore an indication from a small, budget-limited experiment on one dataset, not a ranking.

N. Cost of the detection stage and of the full pipeline

The growth experiment reported later in this section uses only 100 to 900 rows and cannot be extrapolated. I therefore measured the detectors again on a protocol fixed in advance (`verification/cost_study.md`): synthetic Gaussian matrices of 1,000 to 300,000 rows and 8 features, five repetitions up to 30,000 rows and three above, a budget of 300 seconds per measurement, and one fresh process per measurement so that the peak resident memory, native allocations included, belongs to that run alone. The detector is fitted and applied to the same rows, so this is a cost measurement, not a quality one. Times are those of the detection stage (fit, predict and score).

The rule-based detectors and Isolation Forest grow roughly linearly (exponents below one at small sizes reflect fixed start-up costs), LOF grows with an exponent near 1.5 and finished $3 \cdot 10^5$ rows in about 133 seconds, and One-Class SVM, with an exponent near two, exceeded the budget at 10^5 rows and was not tried beyond. The LSTM autoencoder was trained for at most five epochs on at most 50,000 rows, a budget cap: only its scoring cost grows with the input after that. These exponents describe the sizes measured and are not extended beyond them. No measurement was made under contention as defined in the protocol, but the machine was not isolated.

TABLE IX
MEDIAN DETECTION-STAGE TIME ON SYNTHETIC DATA (8 FEATURES), PEAK RESIDENT MEMORY ADDED BY DETECTION AT THE LARGEST SIZE, AND THE EMPIRICAL GROWTH EXPONENT OVER THE SIZES COMPLETED.

Detector	10^4 rows	10^5 rows	$3 \cdot 10^5$ rows	Exponent	Memory (pt. $3 \cdot 10^5$)
Z-score	0.002 s	0.013 s	0.039 s	0.64	70
IQR	0.004 s	0.029 s	0.086 s	0.70	140
Isolation Forest	0.16 s	1.09 s	3.2 s	0.72	115
Autoencoder (MLP)	1.5 s	8.4 s	12.7 s	0.77	499
LSTM autoencoder	1.1 s	5.1 s	7.3 s	0.73	405
LOF	0.73 s	18.0 s	133 s	1.51	788 (pt. $3 \cdot 10^4$)
One-Class SVM	5.6 s	> 300 s	not run	1.99	

To separate the detection stage from the whole pipeline, I also timed read, cleaning, detection and write on the full Credit Card file (284,807 rows read, 283,726 after removing duplicates). With the Z-score the pipeline takes about 4.8 seconds, of which detection is 0.13 seconds (3%) and writing the two sorted files is 3.9 seconds; with Isolation Forest it takes about 8.5 seconds, of which detection is 3.7 seconds (43 to 45%). Peak memory is about 0.75 GB and 0.95 GB respectively. Detection-stage latency is therefore not the latency of the whole pipeline, and the gap depends on the detector. These are single-machine figures on synthetic data and one real file, not a general ranking.

O. Verification outcomes and computation cost

At commit e1fcbb0d (local run, not independently attested) the routine suite runs 1,150 tests, plus three that need the full datasets and twenty-three fault-injection runs, all run separately. Coverage of the pipeline modules and the experiment scripts is complete: 1,628 of 1,628 executable lines and 336 of 336 branches. The tools that check the checks (the evidence and release scripts under `verification/`) are measured apart, at 1,975 of 1,975 lines and 724 of 724 branches, and are not part of that figure. The mutation campaign on the pipeline modules and the experiment scripts kills 4,700 of 4,808 mutants (97.75%). Split by what the code does, the modules that compute a result (splitting, metrics, the synthetic generator, the detectors) kill 1,647 of 1,720 (95.76%) and the input, output and script code kills 3,053 of 3,088 (98.87%), so the aggregate is higher than the score of the scientific computation alone. The 108 survivors count as failures. I classified them by reading the changes, not by proof: most pass an explicit default (`dtype`, `index`) or change an internal label. Two that I first took for unreachable, a cap on a simulated transaction rate, turned out to be reachable by a very rich customer and are now killed by a test.

All twenty-three deliberate defects are caught. The first fourteen were not always caught by the tests I started with: four went through every existing test (a scaler refitted on the scored batch, ranking metrics fed with binary flags, a detector missing from the results, and a contamination threshold read from the synthetic test partition), and a fifth, an inverted Z-score, was caught only by a test added during the mutation work. None of these defects was in the code; the tests were missing. After adding them I reran every experiment: all detection metrics are unchanged and only the timings differ. The nine defects added later (a chronological split that trains on the latest events, validation rows copied into training, an undefined precision reported as one, inverted machine-learning scores, scores reordered before the comparison with the labels, a contamination read from train and test labels, a silently dropped seed, a real-data check that passes without checking the files, and an interrupted mutation campaign accepted on its score) were each caught by a named test. The real-data checks pass locally on the three CSV files, and the six tables of this manuscript match the archived results.

The same profiles were run on a hosted Linux machine for the final revision (commit c64c6ffa, a continuous-integration run, not independently attested). The release script returned *certified*. The routine suite and the scripts keep complete line and branch coverage, the checking scripts are fully covered (1,975 of 1,975 lines), and the mutation campaign, run from scratch, kills 4,703 of 4,808 mutants (97.82%) with no timeout. Earlier hosted runs reported fourteen timeouts, always the same mutants. They were not hangs in the mutated code: the mutation tool starts worker processes after the numerical libraries have created their thread pools, which blocks on Linux. Running the tool with single-threaded libraries removed them, and the same four mutants then gave the same outcomes as on macOS. The three real datasets were downloaded and matched the archived checksums, and the controlled reproduction passed at its three levels. These results show that the evidence is consistent and re-creatable on a second platform; they do not certify the scientific conclusions of the study, which rest on the limits stated with each experiment.

Detector timing in Tables I and III sums `fit`, `predict` and `score` inside the detection stage; CSV loading, deduplication and figure rendering are excluded. A separate local growth experiment ran Isolation Forest, LOF and One-Class SVM on 100, 300 and 900 generated rows, with eight features and three repetitions per size. All 27 runs completed. At 900 rows, median measured detection times were 0.392, 0.020 and 0.049 seconds, respectively, on the recorded macOS/ARM environment. This small in-memory experiment measures neither native-library peak memory nor the scaling of the 284,000-row fraud dataset. It supports reporting the workload and omissions explicitly, but not a general speed ranking.

V. DISCUSSION

a) Rules and learned models are complementary, not competing.: The per-type heatmap shows limited overlap between what the declarative rules catch (missing values), what the statistical rules catch (single-column outliers), and what the learned models add (implausible combinations, recall 0.89 for Isolation Forest against ≤ 0.04 for the rules). Broad coverage needs more than one family, not the single best one on aggregate. This is a different conclusion from benchmarks that publish a single ranking.

b) The operating point matters as much as the algorithm.: Setting Isolation Forest's contamination to the data

prevalence rather than a fixed 0.10 moves its fraud precision from 0.01 to 0.28 (the seed of Table V; 0.20 on average over five seeds) and its multivariate-outlier recall on the synthetic data from about 0.3 to 0.9 (means over five seeds). The same estimator looks weak or strong depending on one number that a benchmark must choose deliberately.

c) Unsupervised detection works in one of the three regimes.: The three real settings bracket the method: on Cardio, where anomalies are rare and distributionally distinct, Isolation Forest reaches $F_1 = 0.62$ and PR-AUC 0.70; on Credit Card Fraud, where the positive class is a 0.17% mixed-defect label, no method clears a usable precision at reasonable recall; on Water Quality, where the "anomaly" is a 61% majority, every method is at chance. The question is less "rules or ML" than "does this dataset have rare, outlying anomalies at all".

d) Aggregate metrics on a mixed label are misleading.: On every dataset the aggregate table compresses different behaviour into similar F_1 and PR-AUC values. A method excellent on one anomaly type and blind to the rest scores like a mediocre all-rounder. The controlled per-type dataset, and the severity sweep that tests the sensitivity of that ranking, are what make the difference measurable.

e) Pipeline integration changes the shortlist.: Latency and scaling matter, but the timing in these runs is too variable to establish a reliable speed ranking. The One-Class SVM and LOF were omitted from the full fraud dataset because their computational costs need a separate scaling study. I time only the detection stage (fit, predict and score) inside the pipeline; extraction, transformation and loading are outside the measurement, so these figures are not the latency of a whole ETL run.

f) Threats to validity.: The synthetic dataset still encodes my own assumptions about what each anomaly type looks like; the severity sweep addresses the magnitude choice but not the shape of the injections. Cardio is a single genuinely-imbalanced dataset, so "the per-type ranking generalises" remains a claim about the synthetic data plus one real point. The ML detectors carry a wide seed spread on the small datasets (Cardio F_1 standard deviation up to 0.17; Isolation Forest on fraud $F_1 = 0.20 \pm 0.07$), so those numbers should be read as ranges. Verification has its own limits: complete line and branch coverage does not establish that the scientific assertions detect every defect. The previous archive was made on a working tree with uncommitted changes (revision `841c3aa`). The archived runs were therefore replayed from a clean commit (`93b6fe8`): every detection metric, curve and partition is identical to the previous archive and only the timings changed, which is why the two timing tables and the scaling figures above were updated. The new records name that commit and mark the working tree as clean, but they were produced on one machine and the timings do not transfer to another. Mutation testing reaches the target on the pipeline modules and the scripts, but it cannot show whether the injected defects and the tests cover every way an experiment can go wrong. The complete real-data checks were performed with the three files available locally. The Water Quality file matches its named mirror, and the Cardio CSV can be reproduced from the current ADBench archive. The Credit Card file has the size of the current Kaggle version (3, March 2018), but I could not compare its content hash with a fresh download. These gaps leave some implementation errors capable of changing a published result without being caught by the current gates.

VI. Conclusion

I built an ETL pipeline in which rule-based and classical machine-learning anomaly detectors share one interface and are scored on the same held-out path, over five seeds, with the operating point read from the data. A controlled dataset labels six anomaly types separately, and a severity sweep shows where the per-type ranking changes as the injections are scaled. On aggregate metrics the detector families stay close on all four datasets, and on 100 labelled problems of a pool of 152 open tables no detector is clearly ahead (Section IV-J). The per-type view is where they diverge: declarative rules own missing values, statistical rules own single-column outliers, tree- and boundary-based models are the only ones that catch implausible combinations, and duplicates and temporal drift need dedicated stages. Across the three real datasets, unsupervised detection pays off only on Cardio, where the anomalies are genuinely rare and outlying. The choice of contamination, the precision-recall trade-off it selects, and the pipeline cost further constrain which detector fits a real flow.

The next steps are the evaluation of the deep-learning detectors behind the same interface (a CPU autoencoder and the causal windows an LSTM-Autoencoder needs are implemented and tested but not yet compared), more genuinely imbalanced real datasets to firm up the per-type generalisation claim, and the evaluation of the windowed drift monitor on real data rather than the per-row detector used here.

References

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

[2] Y. Zhou, F. Tu, K. Sha, J. Ding, and H. Chen, "A survey on data quality dimensions and tools for machine learning," *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614

[3] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, pp. 7920–7927, 2025.

[4] D. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, pp. 53–60, 2024.

[5] M. R. Sokkula and S. K. Vuppala, "Implementing machine learning for ETL data transformation and anomaly detection," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019. [Online]. Available: https://jmlr.org/papers/v20/19-011.html

[8] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] A. Mbata, Y. Sripada, and M. Zhong, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[10] Machine Learning Group – ULB, "Credit card fraud detection," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[11] D. Campos and J. Bernardes, "Cardiotocography," UCI Machine Learning Repository, 2000, names as listed by the repository.

[12] A. Kadiwal, "Water quality," Kaggle dataset, 2021. [Online]. Available: https://www.kaggle.com/datasets/adityakadiwal/water-potability